

Examen Corto 1

Información

Integrantes:

Name	Institution ID	GitHub User
Josué Say	22801	JosueSay
Carlos Valladares	221164	vgcarlol

- [Repositorio](#)

Preparación de entorno

```
In [ ]: # %pip install -r requirements.txt
# jupyter nbconvert se1.ipynb --to html
```

Task 1 - Análisis

Imagine que está diseñando el primer bloque de una arquitectura convolucional para procesar imágenes satelitales multiespectrales de entrada con dimensiones $W = 256$, $H = 256$, $C = 4$ (RGB + Infrarrojo). Tiene dos opciones de diseño para este primer bloque, asumiendo que ambas buscan extraer 64 canales de salida ($C_{out} = 64$):

- **Opción A (Estilo AlexNet):** Una capa convolucional con filtros de 7×7 , stride de 2 y padding de 0.
- **Opción B (Estilo VGG):** Tres capas convolucionales secuenciales con filtros de 3×3 , stride de 1 y padding de 1, seguidas de una capa de Max-Pooling de 2×2 (stride 2).

Responda las siguientes preguntas:

Inciso 1

Utilizando la fórmula espacial vista en clase

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

calcule la dimensión espacial de salida ($W_{out} \times H_{out}$) para el tensor después de la **Opción A** y después de toda la **Opción B** (incluyendo el pooling). Muestre su procedimiento.

Datos de entrada

$$W = 256, \quad H = 256, \quad C = 4$$

Opción A (estilo AlexNet)

Una capa **convolucional** con:

- $K = 7$
- $S = 2$
- $P = 0$

Cálculo del ancho de salida:

$$\begin{aligned} W_{out} &= \left\lfloor \frac{256 - 7 + 2(0)}{2} \right\rfloor + 1 \\ &= \left\lfloor \frac{249}{2} \right\rfloor + 1 \\ &= \lfloor 124.5 \rfloor + 1 \\ &= 124 + 1 \\ &= 125 \end{aligned}$$

Cálculo del alto de salida:

El cálculo es idéntico porque $W = H$:

$$H_{out} = 125$$

Resultado:

$$W_{out} \times H_{out} = 125 \times 125$$

Opción B (estilo VGG)

La arquitectura contiene:

- Tres capas **CONV** 3×3
- $S = 1$
- $P = 1$
- Capa **Max Pooling** 2×2 con $S = 2$

Cada **CONV** 3×3 - Para cada convolución:

$$K = 3, \quad S = 1, \quad P = 1$$

Aplicamos la fórmula:

$$W_1 = \left\lfloor \frac{256 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$= \lfloor 255 \rfloor + 1$$

$$= 256$$

Por lo tanto:

$$256 \times 256$$

Como las tres convoluciones tienen los mismos hiperparámetros el tamaño espacial es el mismo.

Después de CONV1:

$$256 \times 256$$

Después de CONV2:

$$256 \times 256$$

Después de CONV3:

$$256 \times 256$$

MaxPooling 2×2 - Para pooling (sin padding) se usa:

$$O = \left\lfloor \frac{W - F}{S} \right\rfloor + 1$$

Con:

$$F = 2, \quad S = 2$$

Ancho de salida:

$$W_{out} = \left\lfloor \frac{256 - 2}{2} \right\rfloor + 1$$

$$= \left\lfloor \frac{254}{2} \right\rfloor + 1$$

$$= 127 + 1$$

$$= 128$$

Alto de salida:

$$H_{out} = 128$$

Resultado:

$$W_{out} \times H_{out} = 128 \times 128$$

Resultados

Opción A

$$125 \times 125$$

Opción B

$$128 \times 128$$

Inciso 2

Calcula matemáticamente cuántos parámetros entrenables (pesos) requiere la **Opción A** vs. las capas convolucionales de la **Opción B**. Basado en la teoría del **Campo Receptivo**, justifica por qué la industria estandarizó la **Opción B**.

Datos

En una convolución con:

- kernel $K \times K$
- canales de entrada C_{in}
- canales de salida C_{out}

Los **pesos** son:

$$\#W = K^2, C_{in}, C_{out}$$

- Entrada: $C_{in} = 4$
- Salida esperada: $C_{out} = 64$

Opción A (AlexNet) - CONV 7×7

$$\#W_A = 7^2 \cdot 4 \cdot 64$$

$$\#W_A = 49 \cdot 4 \cdot 64$$

$$\#W_A = 196 \cdot 64$$

$$\#W_A = 12,544$$

Requiere 12,544 pesos.

Opción B (VGG) - 3 CONV 3×3

Como el enunciado dice que el bloque "busca extraer 64 canales de salida":

- CONV1: $3 \times 3, 4 \rightarrow 64$
- CONV2: $3 \times 3, 64 \rightarrow 64$
- CONV3: $3 \times 3, 64 \rightarrow 64$

CONV1

$$\#W_{B1} = 3^2 \cdot 4 \cdot 64 = 9 \cdot 4 \cdot 64$$

$$\#W_{B1} = 36 \cdot 64 = 2304$$

CONV2

$$\#W_{B2} = 3^2 \cdot 64 \cdot 64 = 9 \cdot 64 \cdot 64$$

$$\#W_{B2} = 9 \cdot 4096 = 36864$$

CONV3

Es igual a CONV2:

$$\#W_{B3} = 36864$$

Total

$$\#W_B = 2304 + 36864 + 36864 = 76032$$

Requiere 76,032 pesos (solo en las capas CONV).

Comparación

$$\#W_A = 12,544 \quad \text{vs} \quad \#W_B = 76,032$$

Justificación de estandarización

La industria estandariza el patrón tipo VGG (apilar kernels pequeños) por estas razones:

Mismo campo receptivo efectivo, más no-linealidad

Apilar 3 capas 3×3 aumenta el campo receptivo efectivo hasta $\approx 7 \times 7$ pero con diferencias:

- Una sola conv 7×7 es una transformación lineal (antes de la activación).
- Tres conv 3×3 tienen ReLU entre medias, o sea más “doblez” no lineal.

Resultado: más poder expresivo con la misma escala espacial de contexto.

Mejor control del sizing

Al insirtir en los parámetros:

- CONV pequeñas (3×3)
- $S = 1$
- $P = 1$ para preservar tamaño y que el downsampling lo haga POOL (2×2 , $S = 2$)

Eso hace que el diseño sea modular y evita “sizing headaches” (Opción B).

En redes grandes, el ahorro aparece donde importa

Aunque en este bloque aislado B puede tener más parámetros, en arquitecturas profundas:

- Kernels grandes al inicio son costosos, y la estrategia de bloques 3×3 se vuelve una regla sistemática y eficiente, además de que el verdadero monstruo de parámetros históricamente han sido las **FC**.

Conclusión

- Opción B se estandariza porque da campo receptivo grande de forma progresiva, con más no-linealidades, y con un patrón modular que mantiene dimensiones estables y hace el diseño escalable.
- La comparación de parámetros depende del ancho de canales por capa. En tu configuración concreta, B tiene más pesos porque sostiene 64 canales en 3 capas.

Task 2 – Criterio

Una cooperativa agrícola en Guatemala los ha contratado para automatizar la detección de enfermedades en cultivos de mango. El gerente de TI de la cooperativa sugiere usar algoritmos tradicionales (SIFT y HOG combinados con un SVM) porque "son más ligeros y no requieren GPUs". Ustedes, como directores del proyecto de IA, proponen utilizar un enfoque basado en Deep Learning usando Transfer Learning con una arquitectura clásica. Respondan:

Inciso 1

Redacte una justificación técnica (máximo 2 párrafos) dirigida al gerente de TI explicando por qué SIFT/HOG fallarían en un entorno agrícola real frente a variaciones de luz, ángulos y oclusiones, y cómo los "Mapas de Características" de una red profunda solucionan la variabilidad semántica.

Respuesta:

Los métodos tradicionales **SIFT** y **HOG** pertenecen al paradigma de extracción manual de características, donde primero se detectan puntos o gradientes y luego se construyen descriptores numéricos que se entregan a un clasificador como **SVM**. Este enfoque funciona bien cuando las estructuras visuales son relativamente estables, pero en entornos agrícolas reales las hojas presentan **variaciones de iluminación, rotación, escala, perspectiva y oclusiones parciales** (características locales). Bajo estas condiciones, los descriptores manuales capturan patrones rígidos basados en gradientes o texturas locales que pueden cambiar significativamente dependiendo del ángulo de captura, la hora del día o el estado fisiológico de la planta. Como resultado, el sistema tiende a perder consistencia cuando

intenta reconocer síntomas de enfermedad que no aparecen siempre con la misma forma o intensidad.

Las **redes neuronales convolucionales profundas** aprenden automáticamente representaciones visuales jerárquicas mediante **mapas de características**. Las primeras capas aprenden filtros simples que detectan bordes y contrastes, las capas intermedias capturan texturas y patrones visuales más complejos, y las capas profundas integran estas señales para formar representaciones semánticas de objetos o estructuras biológicas. Este aprendizaje jerárquico permite que la red reconozca una enfermedad incluso cuando la hoja cambia de orientación, iluminación o contexto visual. Además, mediante **Transfer Learning** es posible reutilizar redes previamente entrenadas en grandes bases de datos visuales y adaptarlas al cultivo de mango, lo que permite obtener modelos robustos sin necesidad de diseñar manualmente descriptores específicos para cada variación visual del entorno agrícola.

Inciso 2

Si tuviera que elegir estrictamente entre usar la arquitectura **AlexNet** o **VGG-16** para extraer características de las hojas, ¿cuál elegiría y por qué? Justifique su respuesta basándose en la modularidad, la no-linealidad (ReLU) y la capacidad de abstracción de capas profundas.

Respuesta:

Elegiría **VGG-16** porque su arquitectura modular basada en bloques repetidos de convoluciones 3×3 permite construir representaciones visuales de manera progresiva y sistemática. A diferencia de AlexNet, que utiliza filtros de distintos tamaños de forma más empírica (por ejemplo 11×11 , 5×5 , 3×3), VGG mantiene un diseño homogéneo que facilita la extracción jerárquica de patrones visuales. Esta modularidad permite que la red capture estructuras visuales de forma gradual, desde bordes y texturas hasta patrones complejos. Además, VGG-16 es considerablemente más profunda, lo que incrementa la cantidad de no-linealidades introducidas por las funciones de activación $\text{ReLU}(x) = \max(0, x)$ entre capas. Cada convolución adicional permite aprender transformaciones más complejas del espacio de características, incrementando la capacidad de abstracción del modelo. Esto significa que VGG puede construir representaciones más ricas y semánticas de las hojas, identificando patrones sutiles de daño o infección incluso bajo variaciones visuales del entorno agrícola. Por esta razón, las arquitecturas profundas como VGG se volvieron una base sólida para **Transfer Learning**, permitiendo reutilizar representaciones visuales generales y adaptarlas a tareas específicas como la detección de enfermedades en cultivos.

Task 3 – Implementación

Utilizaremos el "Mango Leaf Disease Dataset" disponible en Kaggle. El dataset contiene imágenes de hojas de mango clasificadas en múltiples categorías (sanas y con diversas enfermedades).

Deberá escribir un script en Python (preferiblemente un Jupyter Notebook) utilizando PyTorch o TensorFlow/Keras para resolver este problema mediante **Transfer Learning**.

Debe completar por lo menos los siguientes puntos:

1. **Pipeline de Datos:** Escriba el código para cargar las imágenes. Aplique **Data Augmentation** (rotaciones, recortes aleatorios, flípeos) y normalice los tensores de entrada para que coincidan con la distribución matemática que espera la red pre-entrenada (por ejemplo, media y desviación estándar de ImageNet).
2. **Adaptación de la Arquitectura:**
 - a. Instancie un modelo VGG-16 (o AlexNet) con pesos pre-entrenados en ImageNet.
 - b. Congele (freeze) los parámetros de la sección de extracción de características (capas convolucionales) para retener los filtros de Gabor y texturas ya aprendidas.
 - c. Reemplace y diseñe desde cero el bloque final de clasificación (Capas Totalmente Conectadas / Fully Connected) para que la capa de salida coincida exactamente con el número de clases del dataset de hojas de mango.
3. **Entrenamiento y Backpropagación:** Escriba el bucle de entrenamiento (training loop). Utilice una función de pérdida adecuada para clasificación multiclase (ej. CrossEntropyLoss) y un optimizador (ej. Adam o SGD). Entrene su red (solo las capas nuevas) por lo menos de 5 a 10 épocas.
4. **Evaluación y Reporte:**
 - a. Evalúe el modelo en un conjunto de validación/prueba que el modelo no haya visto.
 - b. Genere un breve reporte Markdown dentro de tu Notebook que incluya:
 - i. Precisión global (Accuracy).
 - ii. Una matriz de confusión.
 - iii. Una conclusión de 3 líneas interpretando si hubo sobreajuste (overfitting) o no, recordando el concepto de Dropout visto en clase.

```
In [ ]: import os, time, copy, random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader, random_split, Subset
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

# Reproducibilidad
```



```

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# Parámetros
IMG_SIZE = 64
BATCH_SIZE = 64 if torch.cuda.is_available() else 32
NUM_WORKERS = 0
PIN_MEMORY = True if torch.cuda.is_available() else False

data_dir = "./archive"

# Normalización ImageNet
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

data_transforms = {
    "train": transforms.Compose([
        transforms.RandomResizedCrop(IMG_SIZE),
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(10),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ]),
    "val": transforms.Compose([
        transforms.Resize(int(IMG_SIZE * 1.15)),
        transforms.CenterCrop(IMG_SIZE),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])
}

# Dataset
base_ds = datasets.ImageFolder(data_dir)

class_names = base_ds.classes
num_classes = len(class_names)

print("Classes:", num_classes, class_names)
print("Total imágenes:", len(base_ds))

# Split train / val
train_size = int(0.8 * len(base_ds))
val_size = len(base_ds) - train_size

train_subset, val_subset = random_split(base_ds, [train_size, val_size])

# Wrapper para transforms

```

```

class SubsetWithTransform(torch.utils.data.Dataset):
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform

    def __len__(self):
        return len(self.subset)

    def __getitem__(self, idx):
        x, y = self.subset[idx]
        if self.transform:
            x = self.transform(x)
        return x, y

train_ds = SubsetWithTransform(train_subset, transform=data_transforms["train"])
val_ds = SubsetWithTransform(val_subset, transform=data_transforms["val"])

# Reducir dataset (para velocidad)
MAX_TRAIN_SAMPLES = 800
MAX_VAL_SAMPLES = 200

if len(train_ds) > MAX_TRAIN_SAMPLES:
    train_ds = Subset(train_ds, range(MAX_TRAIN_SAMPLES))

if len(val_ds) > MAX_VAL_SAMPLES:
    val_ds = Subset(val_ds, range(MAX_VAL_SAMPLES))

print("Train samples:", len(train_ds))
print("Val samples:", len(val_ds))

# DataLoaders
dataloaders = {
    "train": DataLoader(train_ds,
                        batch_size=BATCH_SIZE,
                        shuffle=True,
                        num_workers=NUM_WORKERS,
                        pin_memory=PIN_MEMORY),

    "val": DataLoader(val_ds,
                     batch_size=BATCH_SIZE,
                     shuffle=False,
                     num_workers=NUM_WORKERS,
                     pin_memory=PIN_MEMORY)
}

# Modelo VGG16
model = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)

# Freeze convolucionales
for p in model.features.parameters():
    p.requires_grad = False

# Nuevo classifier
model.classifier = nn.Sequential(
    nn.Linear(25088, 1024),
    nn.ReLU(),

```

```

        nn.Dropout(0.5),
        nn.Linear(1024, 256),
        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(256, num_classes)
    )

model = model.to(device)

print(model.classifier)

# Loss y optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.classifier.parameters(), lr=1e-3)

# Training Loop
def train_model(model, dataloaders, num_epochs=5):

    for epoch in range(num_epochs):

        print(f"\nEpoch {epoch+1}/{num_epochs}")
        print("-"*25)

        for phase in ["train", "val"]:

            if phase == "train":
                model.train()
            else:
                model.eval()

            running_loss = 0
            running_corrects = 0
            total_samples = 0

            for inputs, labels in dataloaders[phase]:

                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == "train"):

                    outputs = model(inputs)
                    loss = criterion(outputs, labels)

                    _, preds = torch.max(outputs, 1)

                    if phase == "train":
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels).item()
                total_samples += inputs.size(0)

```

```

        epoch_loss = running_loss / total_samples
        epoch_acc = running_corrects / total_samples

        print(f"{phase} Loss: {epoch_loss:.4f} | Acc: {epoch_acc:.4f}")

    return model

# -----
# Entrenar
# -----
model = train_model(model, dataloaders, num_epochs=5)

```

Device: cuda

Classes: 8 ['Anthracnose', 'Bacterial Canker', 'Cutting Weevil', 'Die Back', 'Gall Midge', 'Healthy', 'Powdery Mildew', 'Sooty Mould']

Total imágenes: 4000

Train samples: 800

Val samples: 200

c:\Users\cvall\AppData\Local\Programs\Python\Python313\Lib\site-packages\torch\cuda_init__.py:287: UserWarning:
 NVIDIA GeForce RTX 5060 Laptop GPU with CUDA capability sm_120 is not compatible with the current PyTorch installation.
 The current PyTorch install supports CUDA capabilities sm_37 sm_50 sm_60 sm_61 sm_70 sm_75 sm_80 sm_86 sm_90 compute_37.
 If you want to use the NVIDIA GeForce RTX 5060 Laptop GPU GPU with PyTorch, please check the instructions at <https://pytorch.org/get-started/locally/>

```
warnings.warn(
```

```

Sequential(
  (0): Linear(in_features=25088, out_features=1024, bias=True)
  (1): ReLU()
  (2): Dropout(p=0.5, inplace=False)
  (3): Linear(in_features=1024, out_features=256, bias=True)
  (4): ReLU()
  (5): Dropout(p=0.3, inplace=False)
  (6): Linear(in_features=256, out_features=8, bias=True)
)

```

Epoch 1/5

```

-----
train Loss: 1.9659 | Acc: 0.3825
val Loss: 0.7107 | Acc: 0.7450

```

Epoch 2/5

```

-----
train Loss: 1.2552 | Acc: 0.5975
val Loss: 0.5746 | Acc: 0.7850

```

Epoch 3/5

```

-----
train Loss: 1.0061 | Acc: 0.6575
val Loss: 0.4854 | Acc: 0.8550

```

Epoch 4/5

```

-----
train Loss: 0.8028 | Acc: 0.7275
val Loss: 0.3636 | Acc: 0.8750

```

Epoch 5/5

```

-----
train Loss: 0.7783 | Acc: 0.7225
val Loss: 0.3561 | Acc: 0.8450

```

4. Evaluación y Reporte

```

In [2]: def evaluate_and_confusion(model, dataloader, class_names):
        model.eval()
        all_preds, all_labels = [], []

        with torch.no_grad():
            for inputs, labels in dataloader:
                inputs = inputs.to(device, non_blocking=True)
                labels = labels.to(device, non_blocking=True)

                outputs = model(inputs)
                _, preds = torch.max(outputs, 1)

                all_preds.extend(preds.cpu().numpy().tolist())
                all_labels.extend(labels.cpu().numpy().tolist())

        cm = confusion_matrix(all_labels, all_preds)

        # accuracy global

```

```

acc = (np.array(all_preds) == np.array(all_labels)).mean()

# Plot matriz de confusión (sin seaborn)
plt.figure(figsize=(10, 8))
plt.imshow(cm, interpolation="nearest")
plt.title("Matriz de Confusión (Validación)")
plt.colorbar()
ticks = np.arange(len(class_names))
plt.xticks(ticks, class_names, rotation=45, ha="right")
plt.yticks(ticks, class_names)
plt.ylabel("Etiqueta real")
plt.xlabel("Predicción")

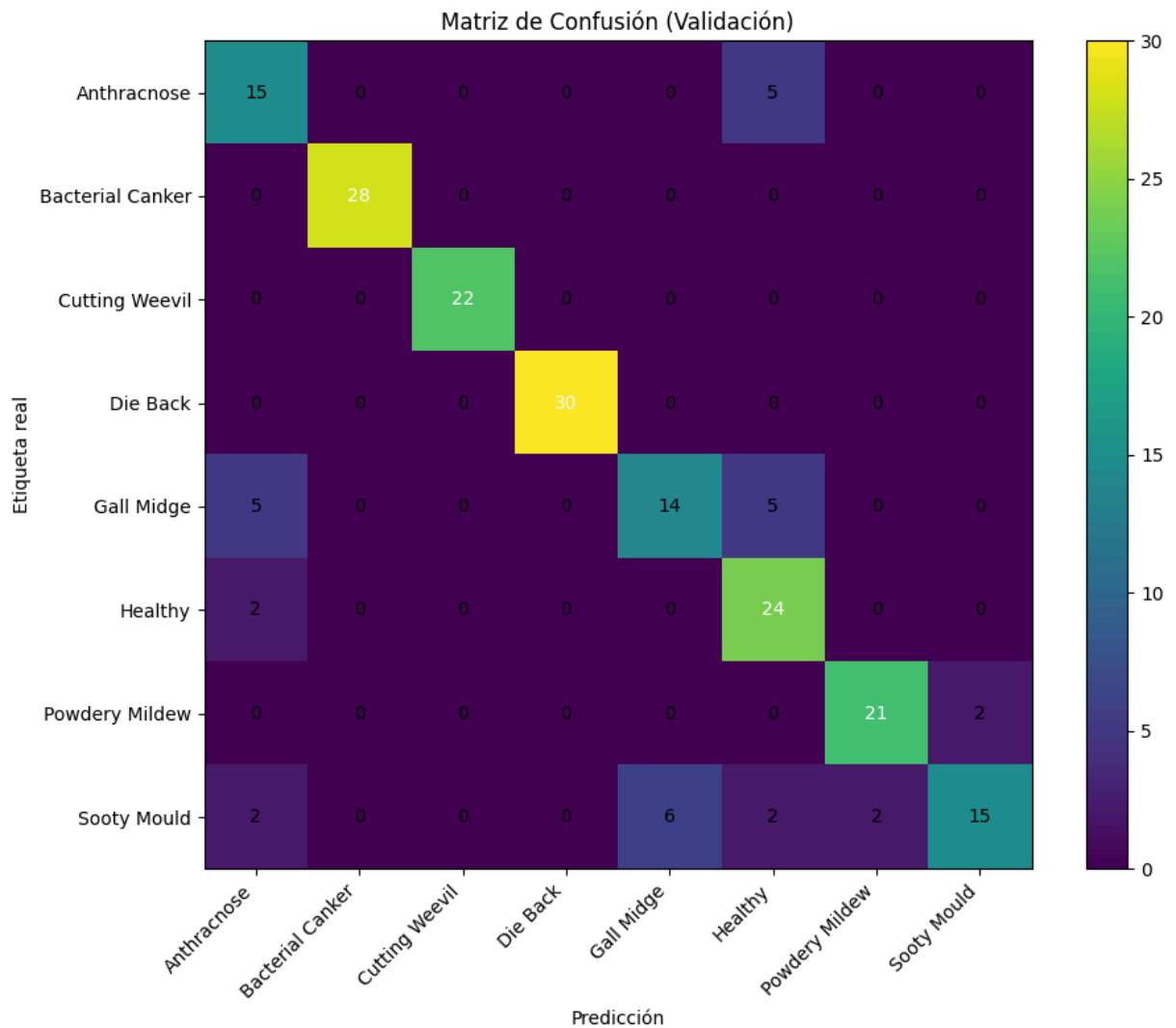
thresh = cm.max()/2.0 if cm.max() else 0.5
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, str(cm[i, j]),
                  ha="center", va="center",
                  color="white" if cm[i, j] > thresh else "black")

plt.tight_layout()
plt.show()

return acc, cm

val_acc, cm = evaluate_and_confusion(model, dataloaders["val"], class_names)
print("Accuracy validación:", val_acc)

```



Accuracy validación: 0.845

Reporte Final

Precisión Global:

Después de aplicar Transfer Learning con VGG-16 preentrenado en ImageNet y entrenar únicamente el clasificador final durante 5 épocas, el modelo obtuvo una **precisión de validación de 0.845 (84.5%)**.

Este resultado indica que los filtros previamente aprendidos por la red para detectar bordes, texturas y patrones generales fueron efectivos para diferenciar varias de las enfermedades presentes en las hojas de mango.

Análisis de la Matriz de Confusión:

La matriz de confusión muestra que varias clases como **Bacterial Canker, Cutting Weevil y Die Back** fueron clasificadas correctamente en la mayoría de los casos.

Sin embargo, se observan algunas confusiones entre clases con características visuales similares, especialmente entre **Gall Midge, Anthracnose y Healthy**, lo cual sugiere que ciertos patrones visuales pueden ser difíciles de diferenciar para el modelo con el entrenamiento limitado.

Conclusión sobre Sobreajuste (Overfitting):

No se observan indicios fuertes de sobreajuste, ya que el modelo generaliza razonablemente bien sobre el conjunto de validación.

El uso de **Dropout en las capas finales** y las técnicas de **Data Augmentation** (rotaciones, recortes y flips) ayudaron a mejorar la capacidad de generalización del modelo.

No obstante, entrenar durante más épocas o utilizar un conjunto de entrenamiento más grande podría mejorar aún más la precisión del sistema.